 ＩＳＢＮ：978-986-02-7580-3

Power-Aware Scheduling for Pinwheel Task Model

Da-Ren Chen[1], Chiun-Chieh Hsu[2] You-Shyang Chen[1] and Shu-Ming Hsieh[3]
[1]Dept. of Information Management Hwa Hsia Institute of Technology
[2]Dept. of Information Management National Taiwan University of science and Technology
[3]Dept. of Computer Science & Information Engineering Hwa Hsia Institute of Technology

{danny, ys_chen, Stanley}@cc.hwh.edu.tw, cchsu@mail.ntust.edu.tw

Abstract

Dynamic voltage scaling (DVS) is a standard technique for controlling the power consumption of embedded systems. A DVS processor can vary its operating voltage and frequency during runtime by using CMOS circuits whose energy consumption has quadratic dependency on the supply voltage. In this paper, we analyze the problem of DVS scheduling for pinwheel tasks based on Distant-Constraint Task Systems (DCTS). We propose a deterministic scheduling algorithm using smooth voltage-adjusting (SVA) technique that reduces many tasks' speeds to a level by sharing the same slack. SVA gathers new scheduling information off-line for distributing the slack to more tasks than other methods do. Additionally, SVA algorithm does not need a canonical schedule whose length usually depends on the least-common-multiplier (LCM) of task periods, thus tasks can be scheduled on-line only with linear time complexity. A large number of experimental results reveal that SVA algorithm outperforms DRA and AGR in energy-saving and time complexity.

1 Introduction

For hard real-time tasks with scarce energy resources, systems are required to reduce power consumption while still meeting the task timing constraints. This can be accomplished by changing processor speed and voltage online, thereby saving energy by spreading run cycles into idle time [15]. Many researchers point out that many processors can run at a range of possible speeds [5,7]. For example, the *Crusoe* processor [16] is able to dynamically change clock frequency in 33 MHz steps and up to 200 frequency/voltage changes per second.

A *slack* is defined as unused processor time due to early completion of a job. Many real-time scheduling algorithms, which reclaim dynamically the available *slack*, must produce a *canonical* (standard) schedule before performing a real schedule. To generate a time interval of such schedule, the time and space complexities depend on the least common multiplier of task periods [2,3,11,12], even if they consider only tasks' arrival/preemption times.

Pinwheel task systems are first motivated by the performance requirements of satellite-based communications [4]. A pinwheel task T_i is defined by two positive integers, an execution requirement and a window length, with explanation that the task T_i need to be allocated to the shared resource for at least a out of every b consecutive time units. Additionally, pinwheel scheduling is also applied in the channel assignment policies with buffer and preemptive priority for RT traffics. In the distance-constrained task systems (DCTS), the temporal distance between any two consecutive executions of a job should always be less than a certain value. In DCTS, pinwheel tasks transform the distance-constraints into 2^n multiples of other shorter periods [8,9], which is not longer than its original distance- constraints by using algorithm ***Sr*** [6]. The advantage of the period transformation is that the produced schedules have regular start, preemption and finish times, and therefore provide good predictability. Suppose, in Figure 2(a), a system have five tasks with distance constraints (periods) {9.2, 10.6, 21.2, 22.6, 23.4}, and their execution times are {1.0, 1.1, 9.98, 0.94, 1,87}, respectively. After applying ***Sr***, the new task periods {5.3, 10.6, 21.2, 21.2, 21.2} are illustrated in Figure 2(b). Because the periods are harmonic numbers, the schedule for each task has no *jitter* [8], and its relative starting and ending times are fixed.

In this paper, we focus on the intertask dynamic voltage scaling algorithms for periodic real-time task systems [2], given that the processor speed can be changed over a continuous range between a pre-defined lower bound and upper bound [2,3,17]. We propose time-efficient and energy-efficient real-time scheduling algorithms, and discuss their performance against other intertask DVS algorithms proposed in [2,3]. The proposed method called Smooth Voltage Adjusting (SVA) can be divided into two parts.

ＩＳＢＮ：978-986-02-7580-3

(1) Due to the period specialization [9], we derive new task parameters from the pinwheel schedule using an off-line algorithm in O(n *log* n) time, where n denotes the number of tasks in the given task set.

(2) When an early completion takes place in a task, the proposed task parameters before this task deadline and unused execution time are utilized by an on-line scheduling. It evaluates the other tasks and decides which jobs of these tasks can properly share the unused execution, and their speed can be reduced to the *same* voltage level. The required time complexity is O(n).

In the conventional schemes [2,3,13,14], the length of a canonical schedule depends on the least-common-multiplier of task periods. However, this is not desirable because the LCM schedule is of exponential length in general, especially when tasks *join* and *leave* the systems frequently. SVA generates necessary tasks information such that the future execution of each task can be predicted rapidly and therefore enables more tasks to share the slack time produced by early-completed tasks. The key motivations of SVA are summarized as follows:

(1) **less frequent voltage scaling and higher utilization of slack:** In SVA, many successive jobs can be reserved for decreasing simultaneously their execution speed to the same voltage/ speed level and therefore the power consumption can be decreased.

(2) **time-efficient technique:** the execution of each task can be profiled quickly without creating a canonical schedule.

(3) **smooth fluctuations of voltage scaling:** the slack will serve as a long time-interval in the schedule as possible for preventing severe fluctuation of voltage scaling.

The remainder of this paper is organized as follows. Section 2 summarizes the background information and the notational conventions. Our power-aware scheduling algorithms are then presented in Section 3. Section 4 includes experimental results, which presents the time-efficiency and energy-efficiency of the proposed real-time scheduling algorithms. Finally, concluding remarks are given in section 5.

2 Task Model

In this section, model assumption and notations are introduced. We focus on synchronous, preemptive and hard real-time task systems; whenever a new task enters the system, all tasks have to be synchronized at a certain time (say t=0). In a periodic task set $\tau = \{T_1, \ldots, T_n\}$ of n periodic tasks, each task T_i consists of a sequence of jobs $J_{i,1}, J_{i,2}, \ldots$. A periodic task T_i with an execution requirement $T_i.e$ and a period $T_i.p$ has *weight* $w_i = T_i.e/T_i.p$, where $0<w_i<1$. The release time and deadline of each task T_i are denoted as r_i and d_i, respectively. Because the first invocation of each task starts at time 0, the release time and deadline of $J_{i,j}$ are derived from $r_{i,j}=(j-1)*T_i.p$ and $d_{i,j}=j*T_i.p$, respectively. In the task model, every task period has been transformed (specialized) as a harmonic number by algorithm ***Sr***[6]. Each task T_i has a unique priority index p_i^* (low values denote high priorities) and is scheduled according to preemptive RM policy. A task set τ contains all tasks in the system and its subsets $\tau_i = \{T_k \mid T_k.p = T_i.p \text{ and } p_k^* < p_i^*\}$, $\tau_i' = \{T_k \mid p_k^* > p_i^*\}$ and $\tau_i'' = \{T_k \mid T_k.p = T_i.p \text{ and } p_k^* > p_i^*\}$. U and U_τ denote the total weights of a task set before and after the *period specialization* [9], respectively. T_{min} and p_{min} denote the highest priority task and the shortest period length among the tasks in τ, respectively. We define the task information of T_i:

***S*$_i$** : actual starting time with respect to r_i.

***E*$_i$** : actual ending time with respect to r_i.

***Fit*$_i$** : the first preemption time with respective to r_i.

re*$_i$*(b,ℓ): the resume execution interval of T_i that starts at time b and is with length of ℓ.

PI*$_i$*(b,ℓ): higher-priority tasks that give rise to preemption interval starting at time b and is with length of ℓ.

***RE*$_i$** : a set of $re_i(b,\ell)$.

***HE*$_i$** : the first resume execution of T_i, i.e., $\{re_i(b,\ell) \mid b=S_i\}$.

***TE*$_i$** : the last resume execution of T_i, i.e., $\{re_i(b,\ell) \mid b+\ell=E_i\}$.

***MaxRE*$_i$**: the longest length of $re_i(b,\ell)$ in RE_i.

***MRE*$_i$**: the shortest length of $re_i(b,\ell)$ in RE_i and excludes the length of TE_i.

***MaxPI*$_i$**: the longest length of PI_i between $[S_i, E_i)$.

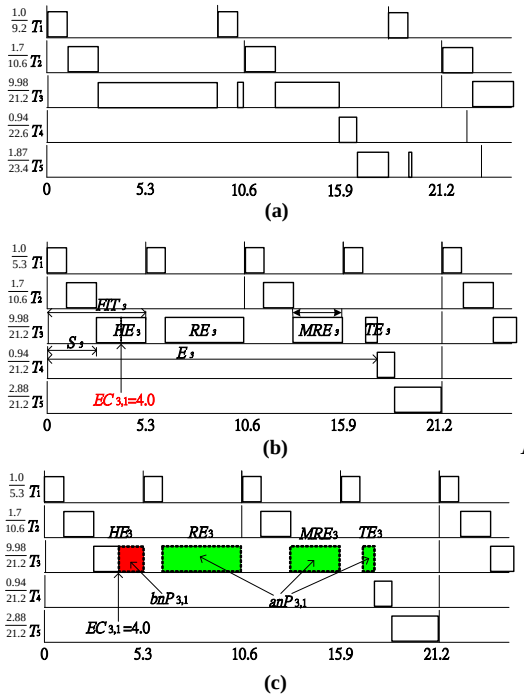

Figure 1. The (a) an original RM schedule (b) DC schedule according to RM policy and (c) schedule produced by SVA algorithm .

As

shown in Figure 1(b), since the lengths of task periods are *harmonic numbers*, each job of task T_i has fixed starting time S_i and ending time E_i with respect to its job release time. During runtime of a task T_i, it could be split into several parts by higher-priority tasks. These parts are classified as *preemption interval* (PI_i), *head execution* (HE_i), *resuming execution* (RE_i) and *tail execution* (TE_i), where TE_i is also the last RE_i in each job. Since the lengths of re_i and PI_i are fixed in each job of T_i, the lengths of $MaxRE_i$, MRE_i and $MaxPI_i$ of the jobs are also fixed.

The power/speed model is similar to those proposed in [3]. The current processor speed of T_i is denoted by ρ_i'. The initial (default) speed of T_i is denoted by ρ_i. In the beginning of every task, the operating system sets $\rho'_i = \rho_i$, prior to any dynamic speed adjustment. $EC_{i,j}$ denotes the early completion time occurred in job $J_{i,j}$. The power consumption of the CPU in the speed ρ_i is given by $P(\rho_i)$, which is assumed to be a strictly increasing convex function denoted by a polynomial function of the third degree. The energy consumed by the processor during the time interval $[t_1, t_2]$ is $E(t_1,t_2)=\int_{t_1}^{t_2} P(\rho(t))dt$. Given that the processor speed can be changed between a minimum speed ρ_{min} and a maximum speed ρ_{max}, where ρ_{min} and ρ_{max} are normalized within interval [0,1) (i.e., $0 \leqq \rho_{min} \leqq \rho_{max} \leqq 1$). In this paper, speed/voltage assignments are determined at task-level [10] and only at task dispatch or completion times.

3 Smooth Voltage Adjusting Scheme (SVA)

Before presenting SVA scheme, the following notations are produced in runtime.

$EC_{i,j}$: the absolute early completion time occurred in job $J_{i,j}$.

$RE\_counter_{i,j}$: the number of re_i that has elapsed before $EC_{i,j}$.

$total\_RE_i$: the total number of re_i in every job of T_i.

$rest\_RE_{i,j}$: the number of the remaining $re_{i,j}$s after $EC_{i,j}$ is at least

$rest\_RE_{i,j} = \max\{0,\ total\_RE_i - RE\_counter_{i,j}\}$.

$rest\_exec^t_{i,j}$: denotes the rest of execution time of $J_{i,j}$ after time t, where $EC_{3,1}=t$.

For example, in Figure 4(b), suppose t=10.6, and $EC_{3,1}$=4.0, $rest\_exec^{10.6}_{3,1}$ =9.98-6.9=3.08.

When $EC_{i,j}=t$, the rest of the slack time before $r_{i,j}+E_{i,j}$ can be split into two parts:

- **before next p_{min}** ($bnP_{i,j}$): denotes the length of time interval $[EC_{i,j},\ \left\lceil \frac{EC_{i,j}}{p_{\min}} \right\rceil \times p_{\min})$.
- **after next p_{min}** ($anP_{i,j}$): denotes the length of slack that falls between $bnP_{i,j}$ and $r_{i,j}+E_i$, which can be defined as $T_i.e - [\left\lceil \frac{EC_{i,j}}{p_{\min}} \right\rceil \times p_{\min} - (r_{i,j}+S_i)]$.

In Figure 1(c), given $EC_{3,1}$=4.0, $RE\_counter_{3,1}$=0 and $rest\_RE_{3,1}$=2, because $EC_{3,1}$ occurred in HE_3. Moreover, $bnP_{3,1}$ denotes the length of [4.0, 5.3], which is 1.3, and $anP_{3,1}$ denotes the rest of the slack in $J_{3,1}$, which can be rewritten as $9.98 - [\left\lceil \frac{4.0}{5.3} \right\rceil \times 5.3 - (0+2.7)]$=7.38. In general, $anP_{i,j}$ could be utilized by the tasks in $\tau - \tau_i' - \tau_i'' - T_i$, and $bnP_{i,j}$ can be applied to the tasks in $\tau_i' \cup \tau_i'' - T_i$. The use of $bnP_{i,j}$ and $anP_{i,j}$ will be discussed in section 3.3 and 3.4, respectively.

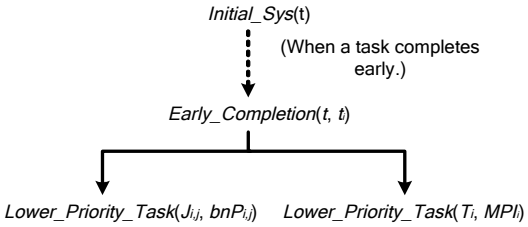

Figure 2. SVA framework.

SVA framework is composed of four independent procedures introduced in the following subsections. In Figure 2, SVA is triggered by early-completed jobs and decreases the speed of other tasks on-the-fly in order to provide additional power savings and meet the task deadlines.

In Figure 3, procedure *reduce_speed*(e^*, L, $\rho_{k,p}$) utilized by SVA can reduce the speed $\rho_{k,p}$ of $J_{k,p} \in \delta$ by allocating L extra units of CPU time for extending the total length of job executions e^*. Thus it decreases the speed of $J_{k,p}$ and is subject to ρ_{min}. This algorithm is quite different from that in [2]. SVA decreases the speed of each $J_{k,p} \in \delta$ to the *same* speed level, where δ denotes a job set containing the feasible jobs varying with time. Consequently, SVA reduces the frequency of speed adjustments, and saves more energy than the previous work does.

Procedure *reduce_speed*(e^*, L, $SP_{k,p}$)
1. Let $S = \frac{e^*}{e^*+L} \times SP_{k,p}$
2. IF $S < SP_{min}$ THEN return SP_{min}
3. return S
End Procedure

Figure 3. Procedure *reduce_speed*.

3.1 *Initialized_Sys*(τ)

The procedure shown in Figure 4 is required in scheduling during system initialization or new tasks arrival, whose time complexity is given in

ＩＳＢＮ：978-986-02-7580-3

Lemma 1. Step3 in this procedure computes relative starting and ending times of each task.

Procedure *Initial_Sys*(τ)
Step 1: Sort the tasks in τ with respect to their periods, such that $T_1.p \le T_2.p \le \cdots \le T_n.p$ and $p_1^* > p_2^* > \ldots > p_n^*$.
Step 2: By algorithm ***Sr*** [8], it transforms the lengths of tasks periods into harmonic number.
Step 3: Computes the S_i and E_i of every task by algorithm *Start_End*.
End Procedure

Figure 4. procedure *Initial_Sys*(τ).

3.2 *Early_Compleltion*(t, τ_i)

When task T_i completes early at time t, SVA performs procedure *Early_Compleltion*(t, τ_i) in Figure 5. Firstly, it computes available slack after time t using Step1 and Step2. It then finds out the task sets that are suitable for sharing the slack using Step3 and Step4.

Procedure *Early_Compleltion*(t, T_i)
Step 1: Computes the lengths of HE_i, MRE_i and TE_i of task T_i .
Step 2: Predicting $rest\_RE_{i,j}$ in $J_{i,j}$.
Step 3: Divide τ into two subsets with respective to p_i^* such that $\tau_i' = \{T_k \mid p_k^* > p_i^*\}$ and $\tau_i''' = \{T_k \mid p_k^* < p_i^*\}$.
Step 4: IF τ_i' is not empty
THEN call *Higher_Priority_Task*(T_i)
IF τ_i''' is not empty
THEN call *Lower_Priority_Task*(T_i)
End Procedure

Figure 5. Procedure *Early_Completion*.

3.3 *Lower_Prioirty_Task*($J_{i,j}$, $bnP_{i,j}$)

Procedure Lower_Priority_Task($J_{i,j}$, $bnP_{i,j}$)
1. Let job set $\delta = \{J_{k,p} \mid T_k \in \tau - \tau_i' - T_i \text{ and } r_{k,p} + S_k > EC_{i,j}\}$.
2. Find all job $J_{k,p}$ such that $J_{k,p} \in \delta$.
3. Let $e^* = \sum_{\forall J_{k,p} \in \delta} T_k.e$.
4. For all $SP_{k,p}' = reduce\_speed(e^*, bnP_{i,j}, SP_{k,p})$
5. For all $T_k'.e = T_k.e \times \frac{SP_{i,j}'}{SP_{i,j}}$.
6. For all $S_{k,p} = S_k - T_k'.e - T_k.e$.
7. Change the start time S_k of $J_{k,p}$ to $S_{k,p}$.
8. Whenever the time is at $S_{k,p}$ and δ is not empty
SET $SP'_k = SP'_{k,p}$

Figure 6. Procedure *Lower_Priority_Task*($J_{i,j}$, $bnP_{i,j}$)

The procedure shown in Figure 6 presents an efficient way to distribute slack $bnP_{i,j}$ among the tasks in $\tau - \tau_i'$.

Notably, $S_{k,p}$ and $\rho_{k,p}$ denote the temporary starting time and the speed of $J_{k,p}$, respectively. The procedure utilizes slack HEs (or $bnP_{k,p}$) and gathers the jobs of which the starting times lie between $EC_{i,j}$ and $r_{i,j}+E_i$. Therefore, the jobs in δ as well as in ready queue have to extend their execution times by shifting their starting times to an earlier time. However, the starting time and the speed of their successor job have to be restored to S_k and ρ_k, respectively.

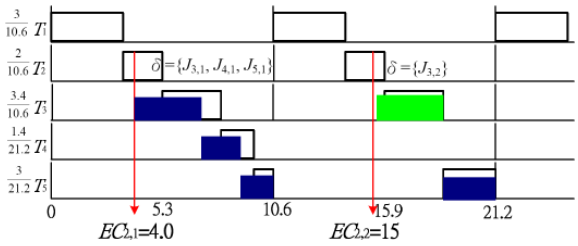

Figure 7. The illustration of *Lower_Priority_Task*( ).

Example. In Figure 7, assuming the default task speeds are 1, S_3=5, S_4=8.4, and S_5=9.8. Given $EC_{2,1}$=4.0, job set δ ={$J_{3,1}$, $J_{4,1}$, $J_{5,1}$}, since $r_{3,1}+S_3$, $r_{4,1}+S_4$ and $r_{5,1}+S_5$ lie between $EC_{2,1}$ and $d_{2,1}$. In line 3, $e^*=T_3.e+T_4.e+T_5.e$=7.8 and $bnP_{3,1}$=1.3, the new speeds of $\rho_{3,1}$, $\rho_{4,1}$ and $\rho_{5,1}$ obtained by *reduce_speed* in line 4 are 0.857. $T'_4.e$ and $T'_5.e$ obtained in line 5 are 3.97, 1.634 and 3.5, respectively. According to line 6 and line 7, the new starting times of $J_{3,1}$, $J_{4,1}$ and $J_{5,1}$ are S_3 — 0.57=4.43, S_4 — 0.234 =8.166, S_5 — 0.5=9.3, respectively. Similarly, $EC_{2,2}$ takes place at time 15, therefore δ ={$J_{3,2}$}. Only the execution of $J_{3,2}$ can be extended, and thus $\rho_{3,2}$=*speed_reduce*(3.4, 0.9, 1)=0.79. In line 5 and line 6, the new starting times of job $J_{3,2}$ are $r_{3,2}+S_3$ — 0.9=10.6+5.9-0.9=15.6. Note that line 8 is triggered whenever a modified starting time $S_{k,p}$ has been reached.

3.4 Higher_Priority_Task

Procedure Higher_Priority_Task($J_{i,j}$, MPI_i)
1. Define a job set $\delta_i^t = \{J_{k,p} \mid T_k \in \tau_i' - T_i, \text{ and } r_{k,p} + S_k \text{ and } r_{k,p} + E_k \in [t,\ t + m \times p_{min}]\}$
2. Let $m = \left\lceil \frac{MPI_i}{P_{min}} \right\rceil$, $\delta_i' = \{\phi\}$.
3. Whenever time t at the beginning of a PI_i and $rest\_RE_{i,j} > 0$.
4. Find the all jobs $J_{k,p} \in \delta_i^t$.
5. $e^* = \sum_{J_{k,p} \in \delta_i'} T_k.e$.
6. For all $\rho_{k,p}' = reduce\_speed(e^*, MRE_i, \rho_{k,p})$
7. For all $T_k'.e = T_k.e \times \frac{\rho_{i,j}'}{\rho_{i,j}}$.
8. For all $E_{k,p} = E_{k-1,j} + T_k'.e$, where $J_{k-1,j} \in \delta_i'$.
9. Change the ending times E_k of $J_{k,p} \in \delta_i'$ to $E_{k,p}$.
10. Whenever time t at the beginning of the p_{min} period where $TE_{i,j}$ locates in.
11. Find the all jobs $J_{k,p} \in \delta_i'$.
12. $e^* = \sum_{J_{k,p} \in \delta_i'} T_k.e$.
13. For all $\rho_{k,p}' = reduce\_speed(e^*, rest\_exec_{i,j}', \rho_{k,p})$
14. For all $T_k'.e = T_k.e \times \frac{\rho_{i,j}'}{\rho_{i,j}}$.
15. For all $E_{k,p} = E_k + T_k'.e - T_k.e$.
16. Change the ending times E_k of $J_{k,p} \in \delta_i'$ to $E_{k,p}$.
End Procedure

Figure 8. Procedure Higher_Priority_Task($J_{i,j}$,PI_i)

When a job $J_{i,j}$ completes early, the procedure distributes slack $anP_{i,j}$ among the tasks having higher priorities than p_i^*. The lengths of HE_i, TE_i, MRE_i and $MaxPI_i$ can be formulated efficiently and the correctness are discussed in Section 4. Let δ_i^t denote the job set consisting of all jobs that release at time t and are eligible for speed reduction by using $anP_{i,j}$. Since $J_{k,p} \in \delta_i^t$ and $r_{k,p}$ is a multiple of p_{min}, we only consider the speed-adjustment at time $t = r_{k,p}$ after $EC_{i,j}$.

ＩＳＢＮ：978-986-02-7580-3

In Figure 8, from line 3 to line 10, it deals with the jobs that release from time t (denotes the multiplier of p_{min}), and their periods do not overlap with $TE_{i,j}$. Since obtaining the actual lengths of every $RE_{i,j}$ in T_i is quite time-wasted, the lengths of $RE_{i,j}$ can be predicted more efficiently by replacing them with MRE_i in line 6. From line 10 to line 16, because the actual length of the rest of $TE_{i,j}$ can be obtained by $rest\_exec_{i,j}^{t}$, the ending time of the jobs in $\delta_{i,j}^{t'}$ can be postponed by sharing $rest\_exec_{i,j}^{t}$. Notably, from line 3 to line 10 in Figure 8, it will be awaked whenever the starting time t has been reached.

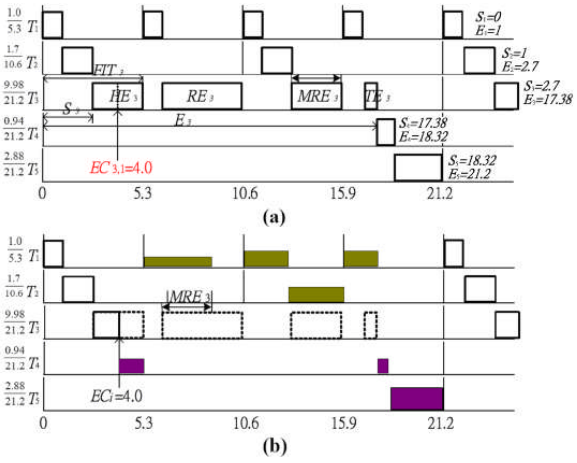

Figure 9. (a)DC schedule according to RM policy and (b)schedule produced by SVA algorithm.

Example. Suppose, in the Figure 9(b), the original task speeds are 1 and $EC_{3,1}$=4.0. The actual lengths of $anP_{3,1}$ can be derived and utilized by jobs $J_{1,2}$, $J_{1,3}$, $J_{1,4}$ and $J_{2,2}$ in procedure *Higher_Priority_Task*. When time t'=5.3, in line 4 of Figure 8, job set $\delta_{3,1}^{5.3}$={$J_{1,2}$}, $r_{1,2}$+$S_{1,2}$ and $r_{1,2}$+$E_{1,2}$ are located in time interval [5.3, 10.6). In line 5, e^*=$T_1.e$=1.0, the new speed $\wp_{1,2}$ derived by the procedure *reduced_speed* in line 6 is $\frac{T_1.e}{T_1.e+MRE_3} = \frac{1.0}{1.0+2.6} = 0.278$, where value 2.6 denotes the length of MRE_3. Since deriving the actual length of every RE_3 increases substantially the scheduling overhead, we use MRE_3 to predict the length of every RE_3 in T_3. In line 7, since $T_1.e$ is 3.60, the new ending time of $J_{1,2}$ obtained in line 7 and line 8 is 3.60. Similarly, at time t'=10.6, we have $\delta_{3,1}^{10.6}$={$J_{1,3}$, $J_{2,2}$}, since $r_{1,3}$+S_1, $r_{2,2}$+S_2, $r_{1,3}$+E_1 and $r_{2,2}$+E_2 are located within [10.6, 15.9]. Therefore, the new speed of $J_{1,3}$ and $J_{2,2}$ is $\frac{1.0+1.7}{1.0+1.7+2.6}$=0.15, and their new ending times are $E_{1,3}$=1.96 and $E_{2,2}$=5.3, respectively. Finally, when time t'=15.9, the jobs beginning during [15.9, 21.2] can share $TE_{3,1}$, which can be derived accurately by $rest\_exec_{3,1}^{15.9}$. In line 11, we obtain $\delta_{3,1}^{15.9}$={$J_{1,4}$} because $r_{1,4}$+S_1 and $r_{1,4}$+E_4 are within the p_{min} where $TE_{3,1}$ locates. The new speed of $J_{1,4}$ derived from *speed_reduce* is $\frac{1.0}{1.0+rest\_exec_{3,1}^{15.9}} = \frac{1.0}{1.0+0.48} = 0.675$. In line 15 and line 16, the new ending time of $J_{1,4}$ is $E_{1,4}$= $\frac{1.0}{0.675} = 1.48$.

4 Experiments

In the experiments, the original task periods are chosen randomly in the interval [2.0~ 10,000.0]ms. Every original task weight has a decimal fraction at most five digits. Before each simulation, the original task periods are specialized according to algorithm ***Sr***, in which the new utilization U_τ is not greater than one. We vary three parameters in our simulations: (1) number of tasks *totaltasks* in τ from 2 to 20, (2) the probability of an early completion (*prob. of EC*) for each job, and (3) the *bc*/*wc* ratio of BCET to WCET, from 0.1 to 0.9. According to the different pair of *totaltasks*, *U* and *bc*/*wc* in *T*, we randomly generate 20,000 task sets. Algorithm ***Sr*** is performed for the 20,000 randomly generated task sets with variable number of tasks in the task sets. In a task set, each task period $T_i.p$ is assigned randomly in the real number range [1, 100]ms with a uniform probability distribution function. The WCET $T_i.e$ of each task is assigned in the real number rang [1, min{$T_i.p$-1, 450}]ms. After giving the values of tasks' periods and executions, we assign the utilization *U* of a task set and rescale the $T_i.p$ of each task such that the summation of the task weights (i.e. $T_i.e$ / $T_i.p$) is equal to a given *U*. The early completion time of each job in simulations (1) and (2) is randomly drawn from a Gaussian distribution in the range of [BCET, WCET], where BC/WC=0.1. In simulation (3), each experiment was performed by varying BCET from 10% to 90% of WCET. After generating the task set, a duplicate of the task sets is transformed by ***Sr***. Therefore, the duplicate has higher actual utilization than that initially intended. In the transformed tasks, since the simulation tasks still complete early proportionately according to the experiment settings, the slack originated from additional utilization can be utilized by slack-time

 ＩＳＢＮ：978-986-02-7580-3

analysis algorithms. Because of the range of the period lengths and the regular nature of pinwheel DCTS, we simulate the schedules of length at most 10,000ms.

The processor model we assumed is base on the ARM8 microprocessor core. For all experiments, we assume there are 10 frequency levels available in the range of 10MHz to 100MHz, with corresponding voltage levels of 1 to 3.3 Volts. The assumption of voltage scaling overhead is the same as that in [1], and an idle processor consumes at most 500μW at the processor sleep mode. The energy consumptions of all the experimental results are normalized against the same processor running at the maximum speed without DVS technique (non-DVS). The minimum speed ρ_{min} is set to 10Mhz, and default speed ρ is set to U_τ. In the test bed, the actual speed ρ^{actual} is selected not less than ρ produced by SVA. We measure the average energy-saving per experiment using a cubic power/speed function. The occurrence of an early-completion taken place randomly between S_i and E_i is controlled by an exponential distribution function, where each job has at most one early-completed event.

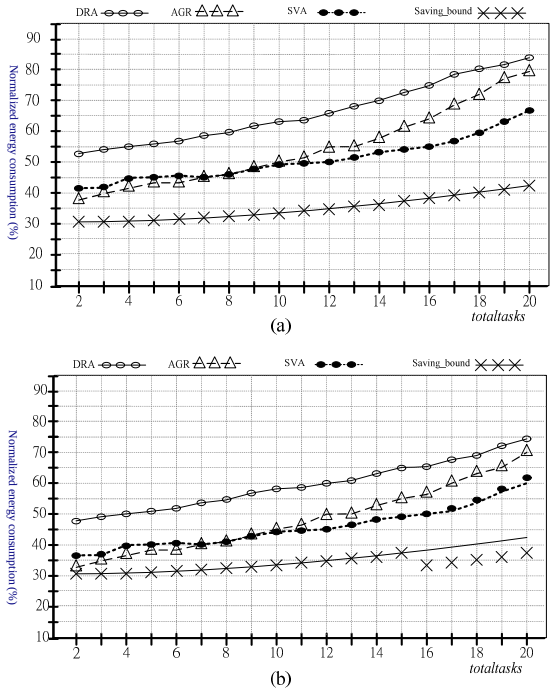

Figure 10. Effect of variations in the *totaltasks* (a) tasks without period transformation and (b) tasks with period transformation (U=60%, bc/wc=0.6)

4.1 Effect of the sizes of τ

Figure 10 presents the energy consumption of the techniques varying with the sizes of τ, when the probability of the early-completion for a job is 0.5. We change the values of *totaltasks* between 2 and 20. We have the following observations:

1. The energy consumption of AGR and DRA is rather insensitive to the variations of *totaltasks*. This is due to the fact that the default speed ρ of a task is very similar to their U_τ, which is generated randomly between $n\times(2^{1/n}-1)$ and 1. Although the number of early completion job increases whenever we increase the value of *totaltasks*, the average generated execution time of a job has to be decreased such that $U_\tau \leqq 1$, which compromises the effect of increased energy saving. However, SVA squeezes further energy saving from the systems with large *totaltasks*. There are two main reasons. Firstly, in procedure *Lower_priority_Task*, when an early completion takes place, it attempts to find the adequate jobs as many as possible and reduces their execution speed simultaneously. Contrary to *Lower_priority_Task*, DRA and AGR could reduce few tasks' speed in the same period of time. Second, because *Higher_priority_Task* further utilizes the unused fragments of slack time (i.e. the *MRE*s), the energy saving produced by our scheme gets closer to its saving bound.
2. The energy consumed by SVA is greater than that of DRA. SVA also outperforms AGR when *totaltasks* is greater than 8. However, from the viewpoint of system implementation, speculating an *aggressive level* in AGR may not be easy. In a real-time system with unstable load, such as radar systems, portal sites and pilot control systems, it cannot predict their average loading at a particular time. Even if the aggressive level can be reached, it must be updated frequently, which results in a large overhead for scheduling systems.

4.2 Effect ACET/WCET and EC

In Figure 11(a), when the *prob. of EC* is smaller than 0.8, SVA performs 2%~12% more energy saving compared to AGR and DRA. When *prob. of EC* is 0.9, the results are not as good as we expect, where there are 1% worse than those of AGR. In the experiments, the *totaltasks* of each task set is randomly determined from 2 to 20. As the probability of early completion decreases, the differences between SVA and DRA or AGR increase substantially. On the contrary, when most of the jobs complete early, the amount of current available slack would be changed frequently and the voltage levels of other related jobs have to be changed. The effects of frequently voltage change compromise the advantage of SVA that benefits the proper distribution of slack. In Figure 11(b), SVA still saves 1% to 10% energy compared to other methods.

ＩＳＢＮ：978-986-02-7580-3

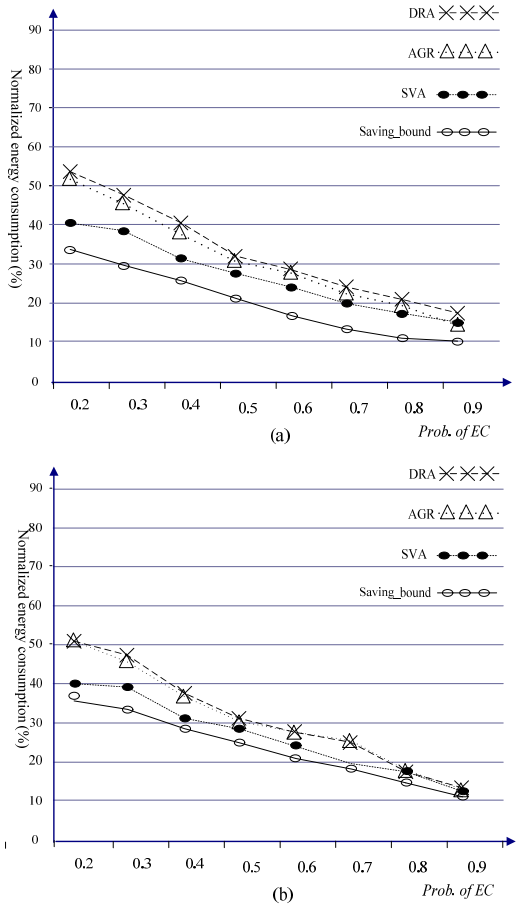

Figure 11. Effect of variations in the *prob. of EC* (a) tasks without period transformation and (b) tasks with period transformation (20 tasks, *U*=60%, *bc/wc*=0.5)

The effect of *bc*/*wc* shown in Figure 12(a) and 12(b) confirms our prediction that the energy consumption (*U*=0.8, *prob. of EC* is 0.5) would be dependent on the variability of the actual workload. When *bc*/*wc*=0.9, the energy consumptions are quite close for all three techniques, as expected. However, once the actual workload continuously decreases, the algorithms are able to reclaim slack time and to save more energy. SVA gives the best energy saving, followed by AGR and DRA. Decreasing the ratio helps further improve the relative performance of SVA because more slack can be distributed properly among a *group* of future jobs. Once we decrease the *bc/wc* ratio less than 0.6, SVA saves greater than 8% energy consumption than those of the other two algorithms. In Figure 12(b), the energy consumption of SVA is closer to DRA and AGR than those in Figure 12(a), because transformed periods improve slack computation of these methods.

In a jitterless schedule, more jobs have the same release times and deadlines than those in the schedules with original periods do. Therefore, the appearances of NTAs become regular and the distances between NTAs are longer than those in the schedules without period transformation. At each scheduling point, when the distance between each pair of consecutive NTAs is lengthened, DRA and AGR obtain more energy savings. In addition, when more tasks release at the same time, we can predict the length of slack more precisely and easier. As the example shown in Figure 9(b), a jitterless schedule makes more jobs share available slack than those with original periods do. Although SVA has the penalty for utilization inflations, it still outperforms other methods.

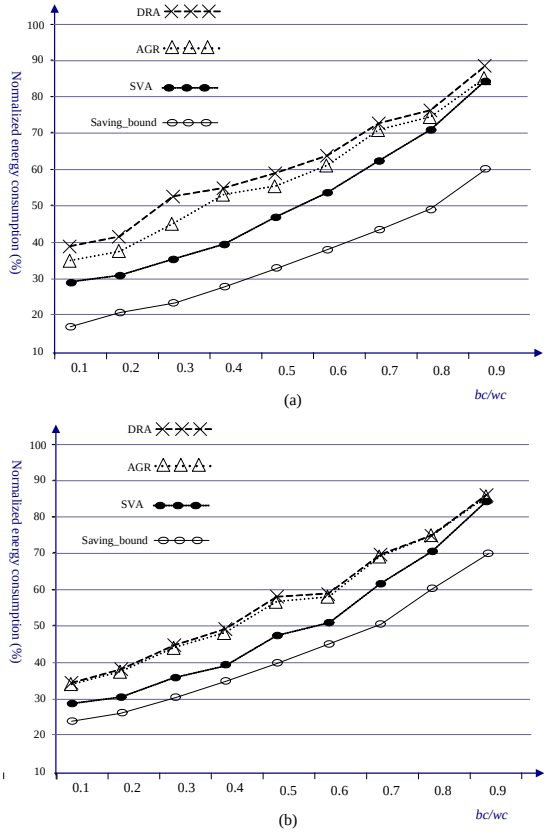

Figure 12. Energy consumption under different *bc/wc* ratio sets (a) tasks without period transformation and (b) tasks with period transformation (2~20 tasks, *U*=60%, *prob. of EC* is 0.5)

5 Conclusion

SVA is a power-aware hard real-time scheduling algorithm with dynamic voltage scaling. This algorithm supports pinwheel tasks based on DCTS model. As an on-line scheduling algorithm, SVA is time-efficient and still achieves significant energy savings. The previous work in the literature [2,3, 8,10,14,16] has to generate a canonical (LCM) schedule before tasks execution. Instead of generating an exponential time schedule in advance, SVA can only attain new parameters of the tasks (*HE*, *TE*, *MRE*, etc.) off-line in O(*n* log *n*) time and performs speed reduction on-line at job's starting

and early-completion times.

In addition, SVA, which agrees with the concept in [15], reduces the fluctuations of task speeds and leads to further energy saving. In Figure 16 and 17, SVA saves up to 30 percent of the energy over the static algorithm. It also offers an advantage over other state-of-the-art intertask voltage scheduling schemes such as DRA and AGR [2]. Finally, SVA outperforms DRA and AGR up to 19% and 20% percent of energy saving in the considerable size of τ.

Acknowledgment

This work was supported in part by the National Science Council of the Republic of China under grant NSC 99–2221–E–146–011.

Reference

[1] ARM 8 Data-Sheet, Document Number ARM DDI0080C, Advanced RISC Machines Ltd, July 1996.

[2] H. Aydin, R. Melhem, D. Mosse, and Pedro Mejia-Avarez, "Power-Aware Scheduling for Periodic Real-Time Tasks," *IEEE Trans. Computers*, Vol. 53, no. 5, pp. 584-600 May 2004.

[3] H. Aydin, R. Melhem, D. Mosse, and Pedro Mejia-Avarez, "Dynamic and Aggressive Power-Aware Scheduling Techniques for Real-Time Systems," *Proceedings of the 22nd IEEE Real-time Systems Symposium (RTSS'01)* . London, England, December 2001.

[4] Sanjoy K. Baruah, Azer Bestavros, "Pinwheel Scheduling for Fault-Tolerant Broadcast Disks in Real-time Database Systems," *Proceedings of the IEEE International Conference on Data Engineering* ICDE 1997: 543-551.

[5] A.P. Chandrakasan, S. Sheng, and R.W. Brodersen, "Low-power CMOS digital design," *IEEE Journal of Solid-State Circuits*, Vol. 27, pp. 473484, Ap. 1992.

[6] C.-C. Han and K.-J. Lin, "Scheduling Distance-Constrained Real-Time Tasks," *In Proc. IEEE Real-Time Systems Symp.*, pp. 300-308, Dec. 1992.

[7] M.A. Horowitz, "Self-clocked structures for low power systems," *ARPA semi-annual report, Computer Systems Laboratory*, Stanford University, Dec. 1993.

[8] C.-W. Hsueh and K.-J. Lin, "Scheduling Real-Time Systems with End-to-End Timing Constraints Using the Distributed Pinwheel Model," *IEEE Trans. Computers*, vol.50, no.1, pp. 51-66, Jan. 2001.

[9] C.-W. Hsueh, K.-J. Lin and C.-J. Hou, "Distance-Constrained Scheduling and Its Applications to Real-Time Systems," *IEEE Trans. Computers*, vol.45, no.7, pp. 814-826, July 1996.

[10] W. Kim, D. Shin, H.S. Yun, J. Kim, and S.L. Min, "Performance Comparison of Dynamic Voltage Scaling Algorithms for Hard Real-Time Systems," *Proc. Eighth Real-Time Technology and Applications Symp.*, 2002.

[11]C.L. Liu, "Scheduling algorithms for multiprocessors in a hard real-time environment, " *JPL Space Programs Summary*, pp.37- 60, 1969.

[12] C.L. Liu and J. Layland, "Scheduling algorithms for multiprogramming in a hard real-time environment, " *J. ACM*, Vol.20, No.1, pp.46-61, 1973.

[13] D. Mosse´, H. Aydin, B. Childers, and R. Melhem, "Compiler-Assisted Dynamic Power-Aware Scheduling for Real-Time Applications," *Proc. Workshop Compilers and Operating Systems for Low-Power* (COLP '00), Oct. 2000.

[14] P. Pillai and K.G. Shin, "Real-Time Dynamic Voltage Scaling for Low Power Embedded Operating Systems," *Proc. 18th Symp. Operating System Principles*, Oct. 2001.

[15] M. Weiser, B. Welch, A. Demers, and S. Shenker, "Scheduling for reduced CPU energy," *Proc. 1st USENIX Symp. on Opemting Systems Design and Implementation*, pp. 13-23, Nov. 1994.

[16] D. Zhu, D. Mosse and R. Melhem, "Power-Aware Scheduling for AND/OR Graphs in Real-Time Systems," *IEEE Trans. Parallel and Distributed Systems*, Vol. 15, no. 9, pp. 849-864 September 2004.

[17] http://www.transmeta.com, 2000.